SECURITY · DOMAIN 7 - ENSURING DATA SECURITY AND COMPLIANCE

Row filters and column masks

Section 7. Two mechanisms, one that the exam guide names and one that superseded it.

SECTION 7 of 10 WEIGHT 10% TOPIC Fine-grained access

admin@sthithapragna.com

Neither mechanism grants access. Both add restrictions on top of privileges you have already granted separately.

A boolean function, evaluated per row

- A **SQL user-defined function returning a boolean**, evaluated against each row at query time.
- Return true and the row is visible. Return false and it is not. The user never sees the unfiltered table.
- Bound to a table with **ALTER TABLE, set row filter**, naming the columns passed to the function.
- Typically the function checks group membership against a value in the row, such as a region or a cost centre.

> ON THE EXAM
>
> The filter runs at query time, so one physical table serves every audience. There is no filtered copy to keep in sync.

A function on one column, with three constraints

- A SQL function taking the column value and returning either the original or a masked version.
- **The return type must match, or be castable to, the column's data type.**
- **Each column can have exactly one mask.**
- A mask can take **other columns as inputs**, so behaviour can vary on several attributes at once.

> ON THE EXAM
>
> That last point is the useful one. A mask on an email column can consult a sensitivity column to decide how much to hide.

It must be a SQL UDF in Unity Catalog

- The function must be **registered in Unity Catalog**. That is what makes it governable and reusable.
- Python or Scala logic **must be wrapped in a SQL UDF**. The SQL function is what you attach.
- So the module 4 cost ladder applies here too. A pure SQL function is cheaper than one calling out to Python.
- This runs on every row of every query against that table, so the cost is not incidental.

> ON THE EXAM
>
> A masking function is the most frequently executed code you will write. Module 4's argument about UDF cost applies with full force.

Table-level against policy-based

	HOW IT WORKS
Table-level	Attached per table with ALTER TABLE. Managed by the table owner.
ABAC policy	Attached at catalog or schema level, matching tables and columns by governed tags.
Coverage	Table-level covers what you configured. ABAC covers new tables automatically as they are tagged.
Who controls it	Table owners can remove their own. ABAC is set by admins and owners cannot override it.

ON THE EXAM

That last row is the governance argument. A control the person being controlled can remove is not much of a control.

ABAC is GA, and the exam guide predates it

- ABAC policies, governed tags and data classification went **generally available in May 2026**.
- The **July 2026 exam guide names only row filters and column masks**, and never mentions policies.
- So answer exam questions with the table-level mechanism, and know that the docs now recommend policies at scale.
- Some regional documentation pages still say Preview or Beta. The blog and the main pages say GA.

> ON THE EXAM
>
> The fourth rename or reposition this course has met that the guide has not absorbed. Teach the examinable answer, name the current one.

Old runtimes fail securely

- **Databricks Runtime below 12.2 LTS does not support row filters or column masks.**
- Those runtimes **fail securely**. Attempting to read such a table returns **no data at all**.
- That is the correct design. The alternative would be silently returning unprotected data.
- ABAC needs more: serverless, or standard and dedicated compute on **DBR 16.4 or above**.

> ON THE EXAM
>
> An empty result set from an old runtime is the protection working, not a bug. Recognise that symptom.

Four documented limitations

- **You cannot apply row-level security or column masks to a view.** Tables only.
- **Iceberg REST catalog and Unity REST APIs cannot read tables with filters or masks.** Delta Lake APIs are unsupported.
- **Path-based access to the files is not supported** on tables with policies.
- **Providers cannot share tables carrying table-level filters or masks** through OpenSharing.

> ON THE EXAM
>
> That last one is a real design constraint. If a table must be shared and masked, the table-level mechanism will not do it.

How policies change the sharing answer

- Tables with **policy-based** filters or masks **can** be shared, unlike table-level ones.
- The condition: the **share owner must be exempt from the policy**, named in its exempting clause.
- Recipients can apply their own filters and masks to **shared tables and foreign tables**.
- Recipients **cannot** apply them to streaming tables or materialized views.

> ON THE EXAM
>
> So the two mechanisms differ on more than convenience. One of them makes a sharing requirement satisfiable and the other does not.

What a pipeline refresh bakes in

- When a pipeline refreshes a materialized view or streaming table, policies are evaluated using the **pipeline owner's or run-as identity**.
- If that identity is subject to a policy, the resulting table **permanently contains masked or filtered data**.
- The fix: add the pipeline owner or run-as identity to the **exempting clause** of every policy on tables the refresh reads.
- Note the word permanently. The masked values are written into the table, not applied at read time.

> ON THE EXAM
>
> Masking applied at write time is not masking. It is data loss, and no downstream grant will recover it.

Who receives masked data, and who does not

- A policy names **who it applies to**, which is the set of users and groups that receive masked or filtered data.
- And **who is exempt**, which is how service identities and stewards keep unrestricted access.
- So the exemption list is a security control in its own right, and belongs under review.
- An unreviewed exemption list is how a policy quietly stops applying to the people who matter.

> ON THE EXAM
>
> Policies are set by admins rather than table owners, which is the separation of duties argument for using them.

Compute requirements and quotas

- Policies need **serverless**, or **standard or dedicated compute on DBR 16.4 or above**.
- Older runtimes **cannot access** tables secured by a policy at all.
- Quotas: **10 policies per catalog**, **10 per schema**, **5 per table**.
- Creating one requires **EXECUTE** on the function and **MANAGE** on, or ownership of, the object.

> ON THE EXAM
>
> Those quotas are low enough to be a design constraint. Policies are meant to be broad and few, not narrow and many.

Dynamic views

- Before filters and masks, the pattern was a **view containing a case expression** checking group membership.
- It works, and it requires users to query the **view** rather than the table, and to be denied access to the table.
- Grant someone the underlying table by mistake and the protection is simply bypassed.
- Filters and masks attach to the **table itself**, so there is no unprotected object to reach around them.

> ON THE EXAM
>
> This is why filters and masks superseded dynamic views. Protection on the table cannot be routed around; protection in a view can.

What to carry into the exam

- **Neither mechanism grants access.** They restrict on top of privileges granted separately.
- Masks: return type must be castable, **one mask per column**, and they may read other columns.
- **Cannot apply to a view.** Providers cannot share tables with table-level filters or masks.
- Runtimes below 12.2 LTS **fail securely**, returning no data.
- **ABAC is GA and recommended at scale**, and the exam guide does not mention it.

> ON THE EXAM
>
> When a scenario needs one rule applied consistently across many tables, that is the policy argument, even if the exam expects the table-level answer.

A team wants regional analysts to see only their own region's rows in a shared orders table. They create a row filter function checking group membership against the region column and attach it to the table.

Analysts report they cannot see the table at all, and receive a permissions error rather than a filtered result.

What is missing?

A The row filter function must be granted EXECUTE to each analyst group.

B The analysts have no SELECT on the table. A row filter restricts access; it does not grant it.

C Row filters require the table to be a view.

D The filter must be applied to each region's rows separately.

◆ PAUSE AND DECIDE

What is missing?

A The row filter function must be granted EXECUTE to each analyst group.

Relevant when creating a policy. It is not what produces a permissions error on the table.

B The analysts have no SELECT on the table. A row filter restricts access; it does not grant it.

Stated directly: these controls add restrictions on top of object-level privileges, which must be granted separately.

C Row filters require the table to be a view.

The opposite. They cannot be applied to views at all.

D The filter must be applied to each region's rows separately.

One filter evaluates every row. There is nothing to apply per region.

ON THE EXAM

The error is the clue. A permissions error means no access at all, not filtered access, and filtering cannot create access.

> A governance team applies ABAC masking policies to columns tagged as personal data across a catalog. A declarative pipeline reads three of those tables and writes a gold materialized view.
>
> Weeks later, analysts with full access to the gold table find the personal data columns are masked there, and no grant restores them.

What happened?

A The gold table inherited the ABAC policy through the pipeline's lineage.

B The refresh evaluated policies as the pipeline's identity, which was subject to them, so masked values were written permanently.

C Materialized views cannot store personal data columns under Unity Catalog.

D The analysts need EXECUTE on the masking function to see unmasked values.

◆ PAUSE AND DECIDE

What happened?

A The gold table inherited the ABAC policy through the pipeline's lineage.

Policies do not propagate through lineage. The masking was written into the data.

B The refresh evaluated policies as the pipeline's identity, which was subject to them, so masked values were written permanently.

Documented behaviour. The fix is to exempt the pipeline owner or run-as identity from those policies.

C Materialized views cannot store personal data columns under Unity Catalog.

Invented restriction.

D The analysts need EXECUTE on the masking function to see unmasked values.

There is nothing to unmask. The original values were never written.

ON THE EXAM

The phrase no grant restores them is the tell. Read-time masking is reversible by permission; write-time masking is not.

> A provider must share a customer table with an external partner. Regulatory requirements mean two columns must be masked for that partner. The table currently has table-level column masks applied with ALTER TABLE.
>
> The share fails to include the table.

What is the explanation and the route forward?

A Providers cannot share tables with table-level column masks. Use ABAC policies, with the share owner exempt.

B The partner must be granted EXECUTE on the masking functions before the share works.

C Column masks only apply to Databricks recipients, so the share must use the open protocol.

D Materialise a masked copy of the table and share that instead, since masking cannot be shared at all.

◆ PAUSE AND DECIDE

What is the explanation and the route forward?

A Providers cannot share tables with table-level column masks. Use ABAC policies, with the share owner exempt.

A stated limitation, and the documented alternative. Policy-based masks can be shared when the share owner is exempt.

B The partner must be granted EXECUTE on the masking functions before the share works.

The recipient does not execute the provider's functions.

C Column masks only apply to Databricks recipients, so the share must use the open protocol.

The protocol is not the constraint. The table-level mechanism is.

D Materialise a masked copy of the table and share that instead, since masking cannot be shared at all.

Works, and creates the duplicate that sharing exists to avoid, plus the masking can be shared via policies.

ON THE EXAM

The two mechanisms differ on capability, not only on convenience. This requirement is satisfiable with one and not the other.

> A masked salary table is queried by two teams. One team, on a modern runtime, sees masked values as expected. Another team, still on Databricks Runtime 11.3, reports that the table appears to be empty.
>
> No errors appear in their query output.

What is happening?

A The older runtime does not support column masks and fails securely, returning no data.

B The masking function is returning null for every row on that runtime.

C The second team lacks SELECT, which presents as an empty table.

D Column masks are evaluated only on serverless compute.

◆ PAUSE AND DECIDE

What is happening?

A The older runtime does not support column masks and fails securely, returning no data.

Runtimes below 12.2 LTS do not support filters or masks and are documented as failing securely.

B The masking function is returning null for every row on that runtime.

That would return rows containing nulls, not an empty table.

C The second team lacks SELECT, which presents as an empty table.

Missing SELECT produces a permissions error, as in scenario 1.

D Column masks are evaluated only on serverless compute.

That is closer to the ABAC requirement, and table-level masks work on classic compute.

ON THE EXAM

Empty and unprotected are the two possible behaviours for an unsupported runtime. Databricks chose empty, which is the right choice.

> An organisation has 900 tables across four catalogs. New tables are created weekly. A compliance requirement says any column containing an email address must be masked for everyone outside the marketing team.
>
> A team proposes writing a masking function and applying it to each table with ALTER TABLE, then adding it to new tables as they appear.

What is the stronger approach?

A The proposal is fine; per-table application gives the most precise control.

B Use an ABAC policy at catalog level matched on a governed tag, so new tables are covered automatically as they are tagged.

C Create a dynamic view over each table and revoke access to the tables.

D Apply one mask at the schema level using ALTER SCHEMA.

◆ PAUSE AND DECIDE

What is the stronger approach?

A The proposal is fine; per-table application gives the most precise control.

Nine hundred manual applications, and it depends on somebody remembering every new table forever.

B Use an ABAC policy at catalog level matched on a governed tag, so new tables are covered automatically as they are tagged.

Exactly the case the docs recommend policies for: consistency across many tables and automatic coverage of new ones.

C Create a dynamic view over each table and revoke access to the tables.

900 views to maintain, and one mistaken grant on a base table bypasses the protection.

D Apply one mask at the schema level using ALTER SCHEMA.

Table-level masks attach to a column on a table. There is no schema-level form of them.

ON THE EXAM

Note that the exam may still expect the table-level answer. This scenario is about knowing why the recommendation moved.

> An engineer needs a mask on a customer identifier column. It must reveal the full value to fraud investigators, the last four characters to support staff, and nothing to everyone else. They also want a second mask on the same column for a separate regulatory rule.
>
> They ask how to layer the two masks.

What should they be told?

A Each column can have exactly one mask. Combine both rules into a single function.

B Apply the second mask to a duplicate of the column.

C Masks are applied in the order they were attached, so two can coexist.

D Use a row filter for the second rule, since filters and masks can be combined.

◆ PAUSE AND DECIDE

What should they be told?

A Each column can have exactly one mask. Combine both rules into a single function.

A documented constraint. One function can express several tiers, and may read other columns to decide.

B Apply the second mask to a duplicate of the column.

Duplicating a sensitive column to work around a constraint creates a second thing to protect.

C Masks are applied in the order they were attached, so two can coexist.

They cannot. One mask per column.

D Use a row filter for the second rule, since filters and masks can be combined.

A row filter removes rows. It cannot express a column-level regulatory rule.

ON THE EXAM

The three-tier requirement is not the problem. One function handles that easily; the constraint is on the number of masks.